Pętla *while*

Zadanie 1.

Napisz kod programu, który będzie pobierał od użytkownika liczby rzeczywiste do momentu wciśnięcia klawisza ‘q’. Następnie program powinien wyświetlić na ekranie ile wśród podanych liczb było liczb mniejszych od 2.

Zadanie 2.

Napisz program, który będzie symulował logowanie się użytkownika do systemu. Zakładamy, że użytkownik ma 5 prób wprowadzenia hasła. Po każdej nieudanej próbie (poza piątą) program powinien wyświetlić komunikat *Błędne hasło!*. Po wprowadzeniu poprawnego hasła program powinien wyświetlić komunikat *Witaj w systemie!* i następnie powinien zakończyć swoje działanie. Po piątej nieudanej próbie logowania program powinien wyświetlić komunikat *Nie ma hasła – nie ma dostępu!* i następnie powinien zakończyć swoje działanie.

Uwaga:

Poprawne hasło powinno zostać zapisane w osobnej zmiennej o nazwie *popr_haslo*.

Zadanie 3.

Jaki będzie efekt wykonania poniższego kodu. Odpowiedź uzasadnij.

```
n = 20
wynik = 0
while n >= 0:
    n -= 1
    if n % 2 == 1:
        continue
    wynik += n
print(wynik)
```

Zadanie 4.

Podany niżej kod przepisz w taki sposób aby zastąpić pętlę *while* pętlą *for*. Efekt wykonania tego kodu nie może ulec zmianie.

```
i = 10
while i >= 1:
    print(i)
    i -= 1
```

Zadanie 5.

Napisz kod programu, za pomocą którego zrealizujesz poruszanie się po kwadratowej planszy 10x10. Zakładamy, że obiekt początkowo znajduje się w punkcie (0, 0). W każdej iteracji pętli należy pobierać od użytkownika informację, w którą stronę (o jedno pole) ma przemieścić się obiekt:

`'g'` - góra

`'d'` - dół

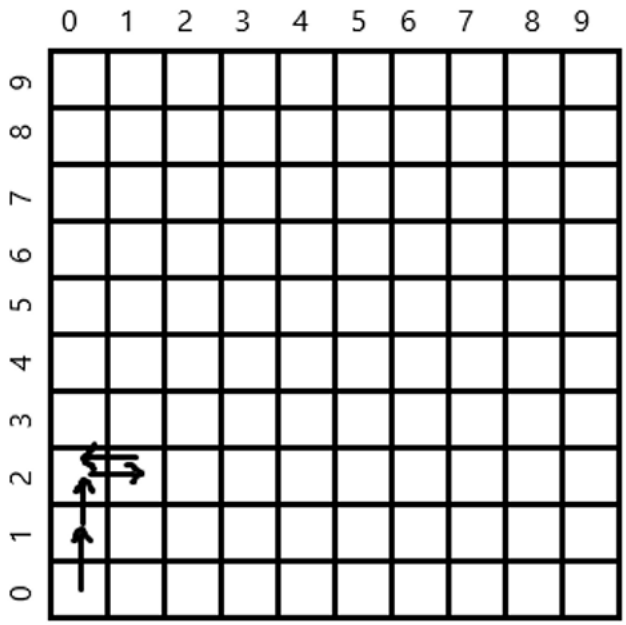
`'p'` - prawo

`'l'` – lewo

Ponadto zakładamy, że obiekt może znajdować się jedynie na polach o współrzędnych $x, y \in \{0,2,\dots,9\}$. Zatem w przypadku gdy wykonanie danego ruchu jest niemożliwe program powinien wyświetlić na ekranie odpowiedni komunikat. Po wykonaniu każdego ruchu na ekranie powinna się pojawiać aktualna pozycja (*x*, *y*) obiektu na planszy. Wciśnięcie klawisza `'q'` powinno kończyć program.

Przykład wykonania programu:

```
Wykonaj ruch
g
(0, 1)
Wykonaj ruch
g
(0, 2)
Wykonaj ruch
p
(1, 2)
Wykonaj ruch
l
(0,2)
Wykonaj ruch
l
Niemożliwe
Wykonaj ruch
q
Koniec
```



Zadanie 6.

Napisz program symulujący przebieg 1 seta w siatkówce plażowej. Przyjmijmy następujące zasady:

- mecz jest rozgrywany przez dwie drużyny dwuosobowe
- w danej akcji punkt zdobywa dokładnie jedna drużyna (ta, która tę akcję wygrała 😊)
- wygrywa drużyna, która zgromadziła przynajmniej 21 punktów i jednocześnie ma ich o 2 więcej niż przeciwnik.

Przykładowy przebieg działania programu:

```
Akcja 1

Podaj nr drużyny, która wygrała akcję.

1

Wynik 1:0

Akcja 2

Podaj nr drużyny, która wygrała akcję.

1

Wynik 2:0

Akcja 3

Podaj nr drużyny, która wygrała akcję.

2

Wynik 2:1

. . .

Akcja 39

Podaj nr drużyny, która wygrała akcję.

2

Wynik 20:19

Akcja 40

Podaj nr drużyny, która wygrała akcję.

1

Wynik 21:19

DRUŻYNA 1 WYGRYWA SETA!
```

Zadanie 7.

Napisz program, który poprosi użytkownika o liczbę całkowitą dodatnią i wypisze ją "od końca" (np. 1234 → 4321). Nie należy używać wbudowanych funkcji odwracania – należy wykorzystać pętlę *while* i instrukcje warunkowe.

Zadanie 8.

Napisz program, który poda dla pobranej od użytkownika liczby całkowitej dodatniej:

a) liczbę czynników pierwszych w jej rozkładzie

b) liczbę różnych czynników pierwszych w jej rozkładzie.

Zadanie 9.

Napisz program, który wylosuje liczbę całkowitą dodatnią z zakresu od 1 do 100 i będzie prosił użytkownika o odgadnięcie tej liczby (tak długo aż użytkownik ją odgadnie). Po podaniu każdej niepoprawnej liczby program powinien wyświetlić na ekranie konsoli jeden z dwóch komunikatów:

- *Podana liczba jest za mała*
- *Podana liczba jest za duża*

Gdy użytkownik poda poprawną liczbę program powinien wyświetlić komunikat:

Brawo! Odgadłeś wylosowaną liczbę.

Ponadto program powinien wyświetlić liczbę prób, które były potrzebne użytkownikowi do odgadnięcia tej liczby (wliczając w to pierwszą próbę) i zakończyć swoje działanie.

Zadanie 10.

Napisz program, który:

1. Pobiera od użytkownika dowolną liczbę całkowitą dodatnią.

2. Oblicza sumę cyfr tej liczby.

3. Wynikową sumę traktuje jako nową liczbę i powtarza proces, aż liczba stanie się jednocyfrowa.

4. Program musi wypisywać kolejne etapy obliczeń.

5. Całość wykonaj WYŁĄCZNIE przy użyciu pętli *while* (bez pętli *for*, bez funkcji rekurencyjnych).